NANYANG
THE INNOVATIVE POLYTECHNIC

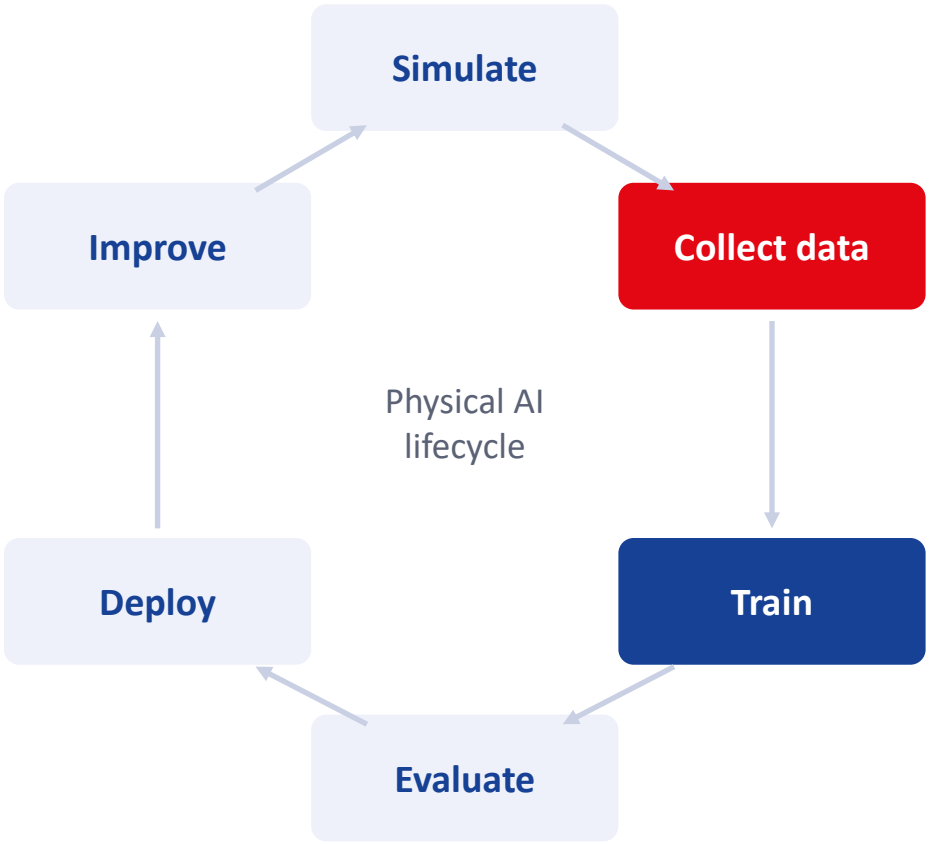
Imitation Learning I

Learning from demonstrations

Data Acquisition & Policy Learning · Lesson 9



Where we are: Block E · imitation learning from demonstrations



A	Foundations L1–L2
B	Collecting robot data L3–L4
C	RL in simulation L5–L6
D	RL: sim → real arm L7–L8
E	**Imitation learning L9–L10**
F	**Evaluate & deploy IL L11–L12**
G	**Vision-language-action L13–L14**

Entry check: three questions on Lessons 7–8

1. In RL, what did the reward do? Did it ever tell the robot which action was right?
2. In Lessons 5–8, what was the human's job? Did a person ever show the arm what to do?
3. If a human demonstrated the task instead, what would replace the reward?

5 minutes, on paper or in the poll. We go through the answers together.

On screen: your pod's 10-episode dataset from Lab 4. Today you will see it is already imitation-learning data.

Learning outcomes

By the end of this lesson you should be able to:

1. **Explain cloning**
   behaviour cloning as supervised learning on observation–action pairs

2. **Name the traps**
   covariate shift, compounding error and multi-modality, and how chunking and coverage help

3. **Describe ACT**
   ACT's inputs, outputs and training needs

4. **Plan the data**
   a 50-episode dataset for your pod's pick-and-place task, in simulation and on the arm

Challenge: write the reward for pick-and-place

Pairs, 60 seconds: write the reward for “pick the cube and place it in the bin”.

Option 1 · write a reward

get the gripper close to the cube
close the gripper at the right moment
lift above a height threshold
carry it towards the bin
open over the bin, and only there
penalties: knocking, flailing, jitter…

…and it may still flick the cube in (Lesson 5).

Option 2 · show it

Hand-guide the leader arm through the task, about 50 times.

The follower records every frame: what the cameras saw, where the joints were, what the arm did next.

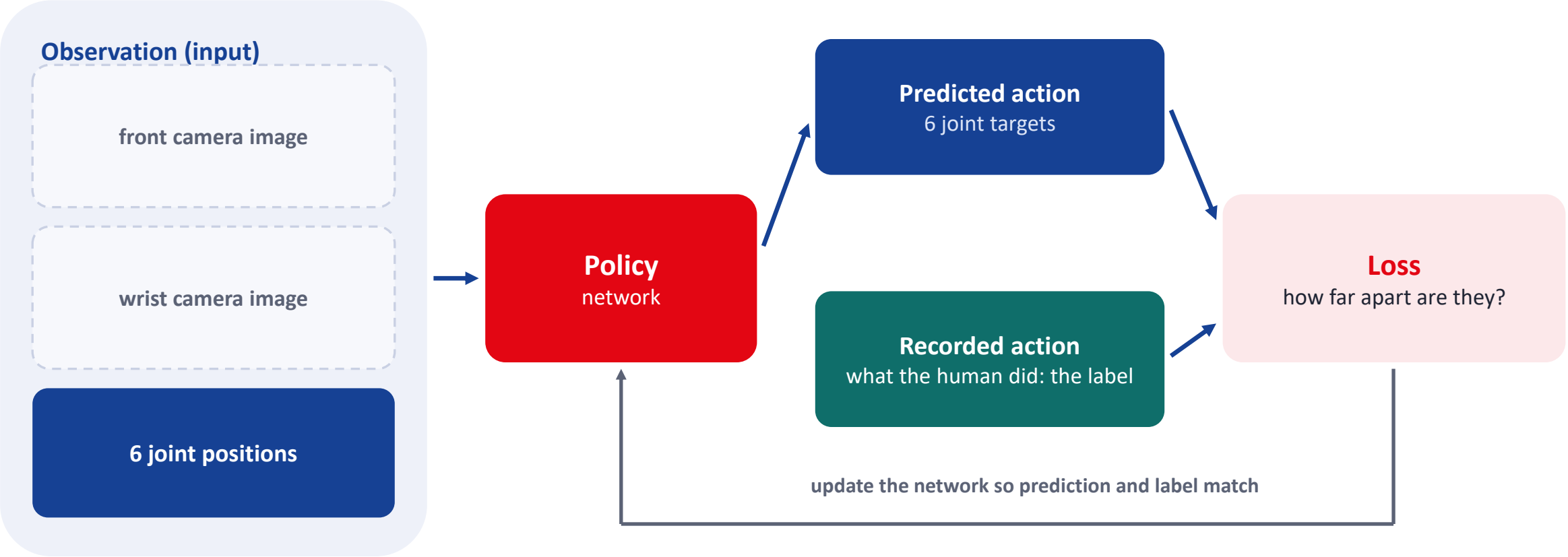
The recorded actions are the answer key. No reward, no simulator, no servo model.

Part 1

From reward to demonstration

Behaviour cloning, and how it compares with RL

Behaviour cloning: supervised learning on robot frames



No reward anywhere: the recorded action is the answer key.

Worked example: the loss on one frame

Joint	Recorded (label)	Predicted	|difference|
shoulder_pan	10.2	11.0	**0.8**
shoulder_lift	48.5	47.0	**1.5**
elbow_flex	−31.0	−30.0	**1.0**
wrist_flex	62.0	60.5	**1.5**
wrist_roll	4.0	4.5	**0.5**
gripper	**35.0**	**30.0**	**5.0**

10.3 ÷ 6

≈ 1.7

L1 loss for this frame

Biggest error: the gripper.

One row in six of the loss, but it decides whether the cube is held.

Training repeats this on every frame: 50 episodes × 20 s × 30 fps = 30,000 labelled examples.

Illustrative values in the arm's calibrated units (joints in degrees, gripper 0–100).

RL and imitation learning, side by side

	Reinforcement learning	Imitation learning
What you write	a reward function	a dataset of demonstrations
Where data comes from	the robot tries things, in simulation	a human shows the task, on the arm or in simulation
Where compute goes	millions of simulated steps, many arms in parallel	hours of supervised training on one GPU
How it fails	reward hacking: it optimises what you wrote	copies mistakes; gets lost where no demo went
What limits it	reward design and the sim-to-real gap	quality and coverage of the demonstrations

In imitation learning, you are the dataset.

Why imitation learning is the practical default on the SO-101

The gap is already in the data

A demonstration recorded on the real arm already contains the real servos, the real delays and the real camera.

Nothing has to be modelled. Nothing has to be randomised.

Backlash

already in every recorded frame:
the follower's lag on every reversal

Friction and load

already in every recorded frame:
how each joint really responds

Delay

already in every recorded frame:
the real loop timing, every frame

Real camera images

already in every recorded frame:
real light, colours and blur

The catch: the policy is only as good as the demonstrations, and only knows places they went. That is Part 2.

Quick poll: which method would you choose?

A

Reach a point you type in, using joint readings only

RL or IL ?

B

Fold a small towel on the table

RL or IL ?

C

Pick a cube from anywhere on the grid and place it in the bin, using the cameras

RL or IL ?

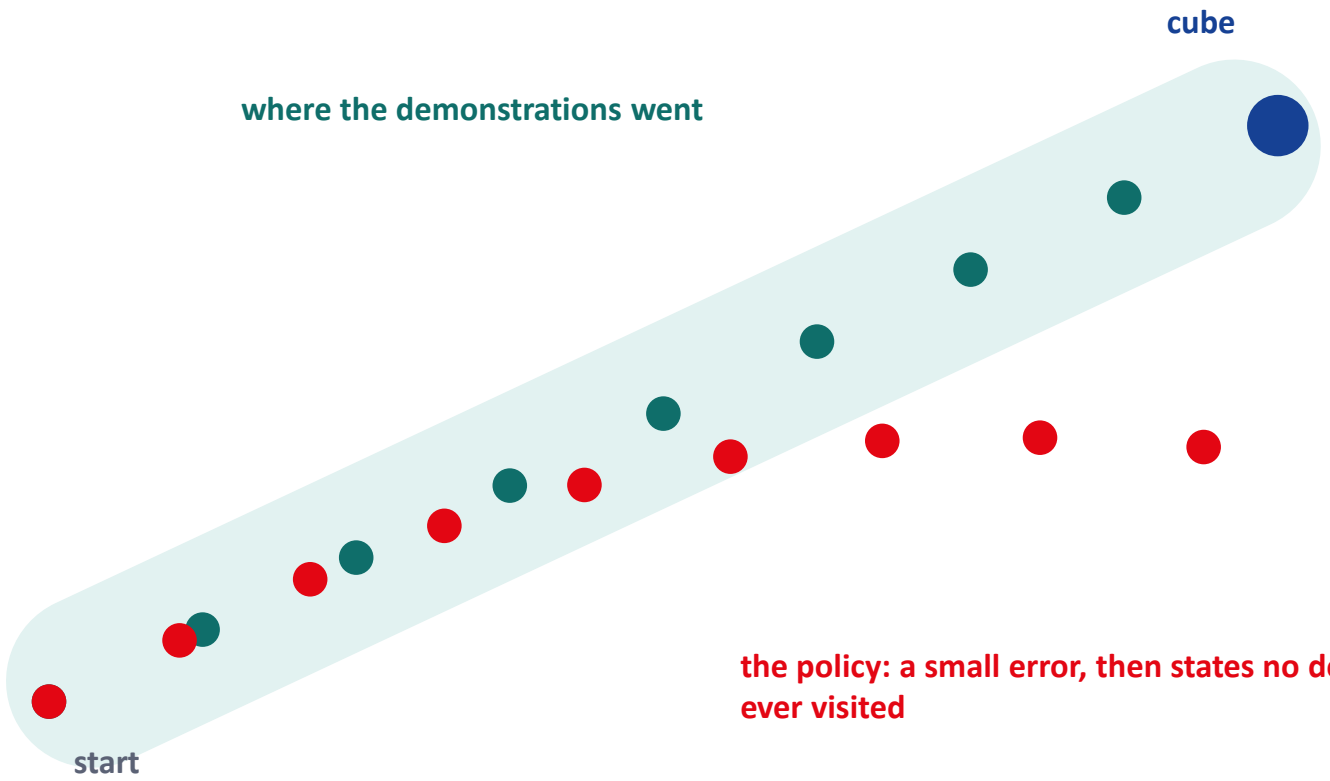
Vote by hand for each task, and be ready to give one reason.

Part 2

Why behaviour cloning is harder than it looks

Covariate shift, compounding error, multi-modality and action chunking

Covariate shift: one small error, then somewhere new



1 · A small error

The arm ends up about 1 cm off the demonstrated path.

2 · Somewhere new

No demonstration passed through this state, so the next action is a guess.

3 · Errors snowball

Each guess moves it further from the data, where guesses get worse.

On a robot, every action changes the next observation. That is why errors compound.

Three fixes: put data where the policy actually goes

1

More coverage

Spread episodes over every starting position you will test: 10 at each of the 5 grid positions.

Effect: the band gets wider

2

Recovery demonstrations

Start a few episodes slightly off the usual path and show the correction back.

Effect: the data shows how to get back

3

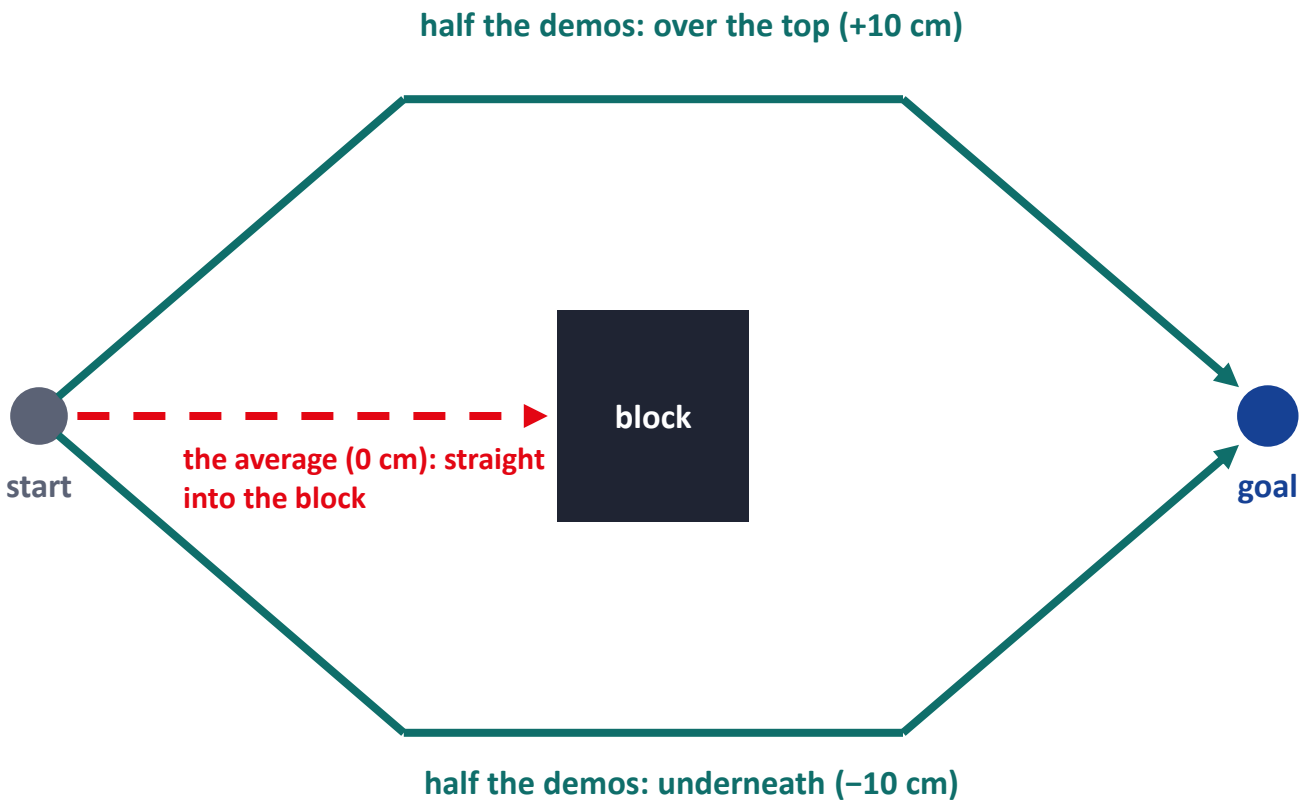
Correction rounds (DAgger)

Let the policy drive; when it drifts, a human takes over. Add those frames and retrain. Used in Lab 12.

Effect: data from the states the policy really visits

All three do the same job: put data where the policy actually goes.

Multi-modality: when the average is a crash



Why it happens

The loss pulls predictions towards the average of the labels. Two good answers average into a bad one.

Fix 1 · the data

One consistent strategy for every demonstration: the L3 rule, and now you know why.

Fix 2 · the model

Policies that can represent several modes: ACT's style variable, Diffusion Policy.

Action chunking: fewer decisions, smoother motion

One decision per step



Chunked: one decision → k actions (k = 6 shown)



Each square is one step at 30 Hz. Red = a fresh decision. Blue = an action taken from the current chunk.

600 decisions

≈ 0.2%

clean episodes

6 decisions (chunks of 100)

≈ 94%

clean episodes

Also

Smoother motion: each chunk is planned as one sequence

Brief pauses in the demos matter less

LeRobot's ACT default: k = 100 (≈ 3.3 s)

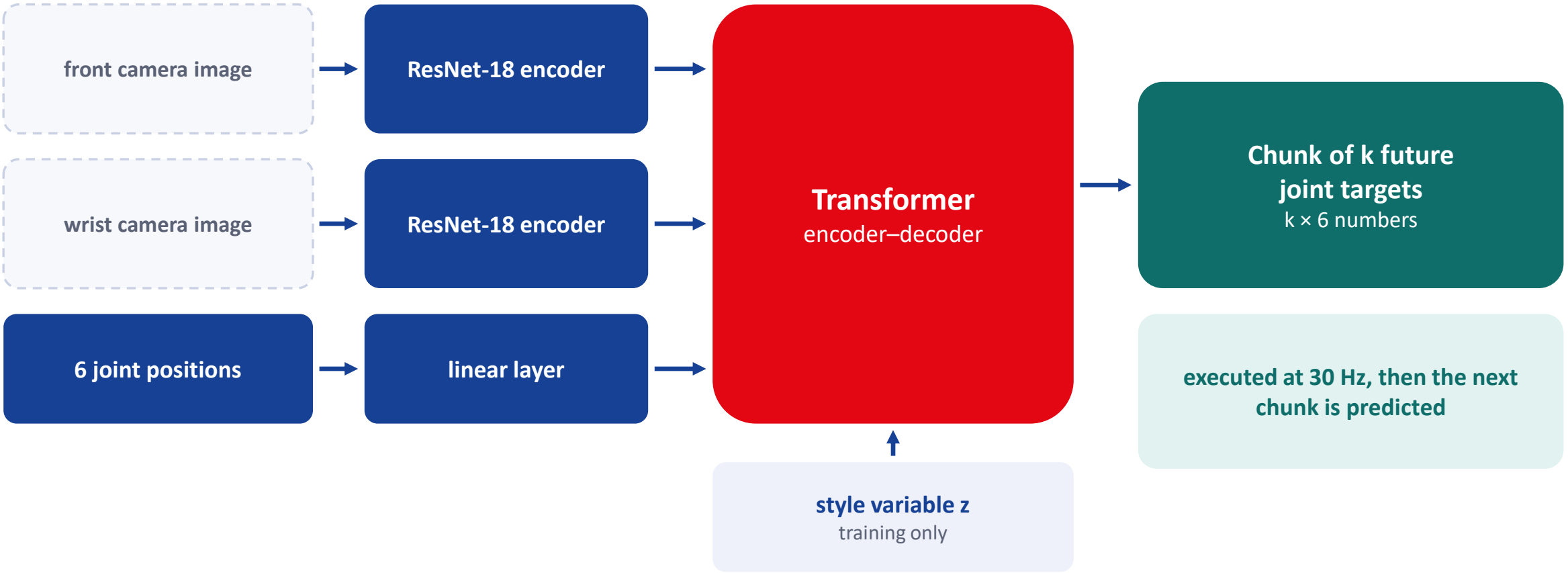
Toy model: every decision has a 1% chance of a derailing mistake. 20 s at 30 Hz = 600 steps. $0.99^{600} \approx 0.002$; $0.99^{6} \approx 0.94$.

Part 3

ACT, and imitation learning from sim to arm

The policy you will train, and how its data gets collected

ACT: Action Chunking with Transformers



Zhao et al. (2023), originally built for low-cost arms. Inputs are exactly what you have recorded since Lab 4.

What ACT needs from you

≈ 50

demonstrations

about 10 at each object position (LeRobot's recommendation)

1 GPU

for training

a single workstation GPU; expect several hours

100

actions per chunk

LeRobot's default, about 3.3 s at 30 Hz

80–90%

success in the paper

six real tasks, from about 10 minutes of demos each

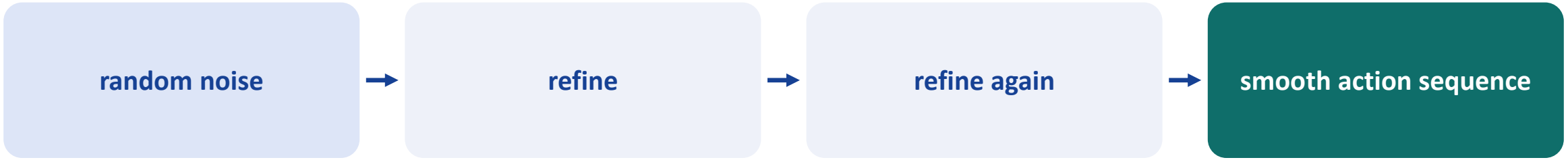
What ACT does not need

a reward, a simulator, or a model of the servos. Everything Block D fought is gone.

What it does need

clean, consistent, well-covered demonstrations. You should be able to do the task by looking only at the camera images.

Diffusion Policy in one slide



Many refinement steps per decision, like an image diffusion model, but the output is a trajectory.

Strength

It can represent several good answers. Over the top or underneath: it commits to one instead of averaging them into a crash.

Cost

Many refinement steps for every decision: more compute to train, and slower to run on the arm.

Concept only in this unit (Chi et al., 2023). ACT is the imitation-learning policy we practise.

Two routes for the data: simulation and the real arm

Simulation route	Leader arm drives the simulated follower	→ Train ACT	→ Check in simulation
Real-arm route	Record leader–follower demos on the real arm	→ Train ACT	→ Deploy on the real arm (Lab 12)

The appearance gap

Simulated images look different from real ones, so a policy trained only in simulation usually stumbles on the real cameras. Use real data, or co-train on simulated + real episodes.

Spot the difference: simulated vs real camera frames

PHOTO
Simulated front-camera frame (Isaac Sim)

PHOTO
Real front-camera frame (your pod)

Lighting and shadows

Table texture

Cube colour

Camera blur and noise

Background clutter

Cables and screws

Pairs, 2 minutes: name three differences a policy might notice. Then compare with the list above.

Training mechanics: steps, checkpoints and the loss curve



Illustrative curve. Your own appears in Lab 10.

Step and batch

One step = one update of the network on one batch of samples picked at random.

Checkpoint

A saved copy of the network. Keep several, not only the last: Lab 12 compares them.

Warning

A low validation loss does not guarantee the arm succeeds. The arm has the final say (Lesson 11).

Worked example: how many times is each frame seen?

50 × 600

episodes × frames each

30,000 frames

100,000 × 8

steps × batch size

800,000 samples

800,000 ÷ 30,000

samples ÷ frames

≈ 27 epochs

Every frame is seen about 27 times, and so is every bad demonstration.

Training cannot tell a good demo from a bad one: it copies both. Curation is the only filter.

Example settings. Read the real steps and batch size in your training config.

Activity: plan your pod's 50-episode dataset

1 · Grid positions

Which 5 positions? How many episodes at each?

2 · Grasp strategy

One sentence every operator follows: approach, side, gripper opening.

3 · Cameras

Where are front and wrist? Is the cube visible at the grasp?

4 · Roles

Operator, scene reset, reviewer. When do you rotate?

5 · Discard rules

Written rules, not feelings: pauses, regrasps, knocks…

6 · Target count

50 kept episodes. How many extra will you record?

Pods, 15 minutes. Hand the plan in today: Lab 10 starts by checking it.

Exit ticket

1. In behaviour cloning, what is the label?
2. Explain covariate shift in two sentences.
3. What does ACT output at each decision?
4. **Pods: hand in your dataset plan.**

On your own, on paper, about 5 minutes.

Next week: L10 lab · collect a quality dataset and train ACT

Arm station · main activity

Record about 50 episodes: 10 at each of the 5 grid positions, one strategy

Rotate roles: operator, scene reset, reviewer

Reviewer marks keep or discard live; re-record discards

Write the datasheet and discard log

Launch ACT training: it keeps running after the lab

Sim station · off-arm time

Record about 10 simulated demonstrations

Compare a simulated frame with a real one and list the differences

Bring: your pod's dataset plan and the calibration files from Lab 2. Gate: curated dataset, datasheet, discard log, and ACT training running.

References and further reading

Zhao, T. Z. et al. (2023). Learning fine-grained bimanual manipulation with low-cost hardware (ACT). https://arxiv.org/abs/2304.13705

Chi, C. et al. (2023). Diffusion Policy: visuomotor policy learning via action diffusion. https://arxiv.org/abs/2303.04137

Ross, S., Gordon, G. & Bagnell, J. A. (2011). A reduction of imitation learning and structured prediction to no-regret online learning (DAgger). https://arxiv.org/abs/1011.0686

Pomerleau, D. (1988). ALVINN: an autonomous land vehicle in a neural network. NeurIPS 1988 (early behaviour cloning).

LeRobot documentation: imitation learning on real-world robots. https://huggingface.co/docs/lerobot/il_robots

LeRobot documentation: ACT policy. https://huggingface.co/docs/lerobot/act

LeIsaac: SO-101 teleoperation and data collection in Isaac Lab. https://github.com/LightwheelAI/leisaac

Advanced topic

From one arm to a humanoid

Behaviour cloning and ACT on humanoid robots: training, inference and what you would need

Extension material: where the method you practise on the SO-101 is used on full humanoid robots.

What stays the same: the SO-101 recipe scales up

1

The label is still the recorded action

A person demonstrates, the robot records what it saw and did, and a network learns to copy. No reward.

2

Covariate shift still bites

A humanoid that drifts off the demonstrated path has to guess, just like the arm. More coverage and recovery demos still fix it.

3

Action chunks still help

Humanoid policies still predict a chunk of future actions: 50 poses in HumanPlus, 48 steps (1.6 s) on Boston Dynamics' Atlas.

4

Curation is still the filter

Boston Dynamics reviews and filters demonstrations before training. A bad demo is copied, whatever the robot.

If you understand today's lesson, you already understand the core of humanoid imitation learning.

What changes: SO-101 arm and humanoid, side by side

	SO-101 arm (this course)	Humanoid robot
Joints to control	6 numbers per step	20 to 50+ numbers per step: 28 on the Unitree H1 in Open-TeleVision
How a person demonstrates	leader arm, the same shape as the robot	VR headset, exoskeleton or body trackers; human and robot bodies differ
Balance	bolted to the table; it cannot fall	must stay upright; a separate balance controller runs underneath
Cameras	front camera + wrist camera	stereo cameras in the head, sometimes wrist cameras too
Data	about 50 episodes for one task	30–40 demos per task, up to hundreds of hours for many tasks
Safety	a light arm on a desk	a person-sized machine: harness, gantry and emergency stop

Same recipe, three new problems: how to demonstrate, how to stay upright, and how much data.

The humanoid imitation pipeline in five steps

1 Teleoperate	2 Retarget	3 Record	4 Train	5 Deploy
A person wears a VR headset or exoskeleton and moves; the robot follows live.	Software converts the person's wrist and finger positions into robot joint angles.	Head-camera images, joint readings and commanded actions, frame by frame.	ACT, Diffusion Policy or a larger VLA model learns: observation → chunk of actions.	The policy moves arms and hands; a balance controller keeps the robot upright.

Familiar: what you do in Lab 10

New for humanoids: covered on the next slides

Steps 3 and 4 are the same LeRobot record-and-train workflow you use on the SO-101.

New problem 1: how does a person demonstrate?

VR headset + hand tracking

The robot's head follows yours and its cameras stream into the headset, so you see what the robot sees. Apple Vision Pro, Meta Quest 3, PICO 4 Ultra.

Exoskeleton

A wearable frame with joint sensors: a leader arm you wear. LeRobot supports the open-source Homunculus exoskeleton for the Unitree G1.

Retargeting: the hidden step

Your arm and the robot's arm have different lengths and joints.

Retargeting software takes where your wrist and fingertips are, and works out the robot joint angles that put its hand in the same place (inverse kinematics).

On the SO-101 you never needed it: the leader arm is the same shape as the follower, so each joint simply copies its partner.

VR + body trackers

Trackers on the chest and feet, plus VR controllers, capture the whole body, including steps. Used for Boston Dynamics' Atlas.

Just one camera

Pose software estimates your body and hands from an ordinary webcam, and the robot copies the pose. Cheapest, least precise. Used in HumanPlus.

New problem 2: staying upright, by splitting the job into layers

Imitation policy (ACT or similar)
learned from demonstrations · decides what the hands do · about 25–30 times a second

arm and hand targets, plus a walking command

Balance / whole-body controller
usually trained with RL in simulation · keeps the robot upright · about 50 times a second

joint targets for the whole body

Motor controllers
hold each joint at its target · up to about 1,000 times a second

Why split the job?

A person in VR can show good hand skills but cannot feel the robot's balance. Balance needs fast, precise reactions, which RL learns well in simulation.

Real examples

HumanPlus: ACT-style policy 25 Hz → RL shadowing policy 50 Hz → motors 1,000 Hz.

LeRobot on the Unitree G1: your policy + a ready-made locomotion controller.

RL and imitation learning are teammates: RL keeps the robot standing, imitation teaches the hands.

Case study 1 · Open-TeleVision: ACT on a humanoid

1 · Collect

Robots: Unitree H1 with dexterous hands, and Fourier GR-1.

The operator wears an Apple Vision Pro. The robot's neck follows the operator's head.

Its stereo camera streams into the headset at 60 Hz; hands and wrists are retargeted.

2 · Train

ACT, with two changes: a stronger image encoder (DINOv2 instead of ResNet-18) and a stereo pair of head images.

Output: a chunk of 28 joint targets per step on the H1 (7 per arm, 6 per hand, 2 neck).

Trained on one RTX 4090 GPU.

3 · Results

Can sorting on the H1: 92% picked, 88% placed.

Towel folding and unloading: 100% in the paper's tests.

Bonus: an operator in Boston controlled a robot in San Diego.

Compared with your SO-101: 6 numbers became 28, a leader arm became a headset, and the policy is still ACT.

Case study 2 · HumanPlus: shadow a person, then imitate

1

Learn to shadow (RL, sim)

A shadowing policy learns to copy 40 hours of recorded human motion in simulation, then moves to the real robot.

2

Shadow a person

One webcam watches the operator; pose software estimates body and hands; the robot copies in real time.

3

Record demos

30–40 demonstrations per task, seen through the robot's two head cameras.

4

Train HIT (ACT-style)

A transformer adapted from ACT predicts a chunk of 50 future poses from two images and joint readings.

Results on a Unitree H1 (33 joints, including hands)

Fold clothes: 100% (40 demos)

Type “AI” on a keyboard: 80% (30 demos)

Put on a shoe and walk: 60% (40 demos)

The lesson for you

The skill policy is ACT-style behaviour cloning on about 40 demos, close to your pod's 50. RL made the body controllable; imitation taught the skill.

Case study 3 · Industry scale: the same idea with far more data

Boston Dynamics Atlas + TRI

450M

parameter policy

VR + chest and foot trackers for whole-body demos.

A diffusion-style policy predicts 48-step chunks (1.6 s) at 30 Hz from head cameras, joints and a language prompt.

When it failed, they recorded recovery demos and retrained.

Figure Helix

≈500 h

of teleoperation data

Two systems: a vision-language model ‘thinks’ at 7–9 Hz; a fast policy ‘acts’ at 200 Hz.

Controls 35 upper-body joints, including fingers.

Runs on embedded GPUs on the robot.

NVIDIA GR00T N1

750K

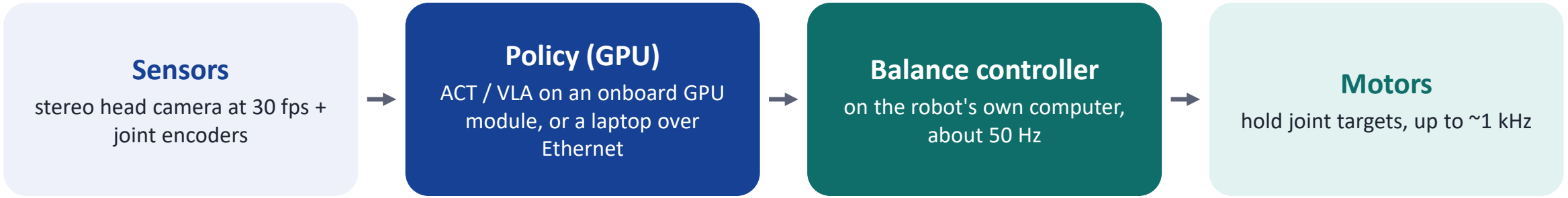
synthetic demos in 11 hours

An open humanoid foundation model trained on a ‘data pyramid’: human videos, simulation and real teleoperation.

You fine-tune it with your own small dataset.

Bigger models, more data, plus language: this is where Lessons 13–14 (vision-language-action) go next.

Inference on a humanoid: what runs where



Each new chunk starts the loop again, exactly like ACT on your arm, just with more layers in between.

Keep the motion smooth

Compute the next chunk while the current one is still running, so the robot never pauses. LeRobot calls this real-time chunking.

Keep people safe

Test in simulation first, then on a harness or gantry, at reduced speed, with an emergency stop within reach.

Speed up carefully

Atlas ran some policies 1.5–2× faster than the demonstrations by replaying each chunk faster, without retraining.

Implementation options: hardware and software

Tier 1 · Simulation only

start here, with what you already have

Hardware: an NVIDIA RTX PC like our Isaac Sim machines; optionally a VR headset.

Software: Isaac Lab humanoid imitation examples (Fourier GR-1 pick-and-place, Unitree G1 loco-manipulation). Isaac Lab Mimic turns about 5 human demos into 1,000. LeRobot also has a MuJoCo Unitree G1.

Tier 2 · Research lab humanoid

a realistic polytechnic or university setup

Hardware: Unitree G1 EDU with dexterous hands (base G1 from US$13.5K; EDU priced on request); Apple Vision Pro, Meta Quest 3 or PICO 4 Ultra; one RTX 4090-class GPU.

Software: Unitree xr_teleoperate to record → unitree_IL_lerobot or LeRobot to train ACT, Diffusion, π0 or GR00T → run on the robot.

Tier 3 · Industrial

company scale

Hardware: full-size humanoids (Atlas, Figure, Fourier GR series), teams of teleoperators, GPUs inside the robot, GPU clusters for training.

Software: in-house foundation models (Helix, Large Behavior Models) or open ones fine-tuned (GR00T N1, π0.5).

The common thread: LeRobot-format datasets and ACT, Diffusion or VLA policies, the same tools you use on the SO-101.

Discussion: think like a humanoid engineer

1. A humanoid must carry a box across a room. Which part would you teach by demonstration, and which by RL? Why?
2. Open-TeleVision streams the robot's own head camera into the operator's headset. Why does that give better training data?
3. HumanPlus needed about 40 demos to fold clothes; Figure collected about 500 hours of data. Why such a big difference?
4. **Your pod's pick-and-place task on a Unitree G1: list the hardware and software you would need for each of the five pipeline steps.**

Pairs, 10 minutes. Be ready to share one answer with the class.

References: advanced topic (humanoids)

Cheng, X. et al. (2024). Open-TeleVision: teleoperation with immersive active visual feedback. https://arxiv.org/abs/2407.01512

Fu, Z. et al. (2024). HumanPlus: humanoid shadowing and imitation from humans. https://arxiv.org/abs/2406.10454

Boston Dynamics (2025). Large behavior models and Atlas find new footing. https://bostondynamics.com/blog/large-behavior-models-atlas-find-new-footing/

Figure AI (2025). Helix: a vision-language-action model for generalist humanoid control. https://www.figure.ai/news/helix

NVIDIA et al. (2025). GR00T N1: an open foundation model for generalist humanoid robots. https://arxiv.org/abs/2503.14734

LeRobot documentation: Unitree G1. https://huggingface.co/docs/lerobot/unitree_g1

Unitree: xr_teleoperate and unitree_IL_lerobot. https://github.com/unitreerobotics/xr_teleoperate · https://github.com/unitreerobotics/unitree_IL_lerobot

Isaac Lab documentation: data generation and imitation learning for humanoids. https://isaac-sim.github.io/IsaacLab/main/source/features/imitation-learning/humanoids_imitation.html